

Name/NetID: Nathan Kariuki / nkari011

Session Date: May 26, 2026 (1.5 hours)

Objective: Implement parallel incremental SSSP and main.cpp together.

Attempts made:

sssp.h reuses the push-style parallel BSP from phase 2. Each round partitions the frontier across TQ worker threads. The version filter is inside the scan loop:

```
shared_lock<shared_mutex> lk(g.locks[u]);
for (const Edge& e : g.adj[u]) {
    if (e.version >= q_version) continue;
    if (dist[u] + e.weight < dist[e.dst])
        dist[e.dst] = dist[u] + e.weight;
}
```

Even if an updater has already appended edges with higher versions by the time we scan, we skip them. The query sees exactly the graph state it's supposed to see.

In main.cpp the producer walks the unified record list and maintains a pending_dirty vector between queries. At each Q it moves pending_dirty into the QueryReq. Seeded pending_dirty with {0} so the first query always has the source vertex in its frontier.

First run hit a segfault and vertices showed as 1. Took a while to figure out — initial.txt has comment lines at the top starting with # that load_initial was reading as edges and getting garbage vertex IDs.

Solutions/Workarounds:

Fixed by skipping comment lines in load_initial:

```
if (line.empty() || line[0] == '#') continue;
```

After the fix vertices showed 4847571 and the run completed.

Commits: main.cpp and loader.h: unified record list, correct producer loop;
fix initial.txt comment skipping, verify both modes correct

Experiments/Analysis:

After fixing the segfault ran ./main 2 2 and got wall time of 13707.1 ms. Ran diff queries.out expected_queries.txt and got no output meaning the files matched exactly.

Learning outcome:

The version filter on edges is what makes the concurrent reads safe without extra locking. Edges already written to the vector but with version $\geq q$ are just invisible to that query, so the SSSP result is always deterministic.

